

System Design Document

Healthcare Appointment & Follow-up Manager

The Healthcare Appointment & Follow-up Manager is designed as a scalable, AI-powered web application that enables patients to book appointments, doctors to manage consultations, and administrators to oversee healthcare operations. The system integrates Large Language Models (LLMs), email notifications, medication reminders, and Google Calendar synchronization while ensuring reliability, security, and high availability.

Double Booking Prevention

To prevent multiple patients from reserving the same appointment slot, the system performs availability verification within a database transaction before confirming a booking. Each appointment is uniquely identified by the doctor, appointment date, and time slot. A unique database constraint ensures that only one confirmed appointment can exist for a specific doctor and time slot. If two users attempt to book simultaneously, the first successful transaction commits while subsequent requests receive a "Slot Already Booked" response without affecting system consistency.

Slot Hold Mechanism

During appointment selection, the requested slot is temporarily marked as **On Hold** for a limited period (for example, five minutes). This reservation allows the patient to complete symptom submission and booking confirmation without another user claiming the same slot. If the booking is completed within the hold period, the status changes to **Confirmed**. Otherwise, the hold expires automatically through a scheduled background task, making the slot available again.

Doctor Leave Conflict Handling

Administrators can define doctor leave dates through the admin portal. Before approving leave, the system checks for existing appointments on the selected date. If appointments exist, affected patients receive notification emails requesting them to reschedule. The appointment status is updated to **Needs Rescheduling**, preventing accidental deletion while preserving appointment history. Available alternative slots are displayed to patients for convenient rebooking.

Notification Retry Strategy

The application uses asynchronous background processing for email notifications. If email delivery fails because of temporary network issues or service outages, the notification is placed into a retry queue. Exponential backoff is applied between retry attempts, and each failure is recorded for monitoring. After the maximum retry limit is reached, the notification status changes to **Failed**, allowing administrators to review unsuccessful deliveries without affecting the primary booking workflow.

LLM Failure Handling

The platform integrates an LLM to generate pre-visit symptom summaries and post-visit patient-friendly consultation summaries. Since external AI services may occasionally fail or exceed response limits, all LLM requests are wrapped in exception handling. If AI generation fails, the appointment proceeds normally while a default message such as "AI summary unavailable at this time" is stored in the database. This ensures uninterrupted healthcare operations even during external service interruptions.

Background Jobs

Several system operations execute asynchronously using APScheduler or Celery workers. These include medication reminders, appointment reminder emails, notification retries, expired slot cleanup, and periodic maintenance tasks. Separating long-running processes from user requests improves responsiveness and prevents delays during appointment booking.

Database Design

The application uses PostgreSQL as the primary relational database. Core entities include Users, Patients, Doctors, Appointments, Symptoms, Prescriptions, Medication Reminders, Notifications, and Doctor Leaves. Foreign key relationships maintain referential integrity, while indexes on doctor identifiers, appointment dates, and slot times enable fast availability searches. Database constraints ensure consistent data and eliminate duplicate bookings.

Scalability

The system follows a modular three-tier architecture consisting of a Next.js frontend, FastAPI backend, and PostgreSQL database. Business logic is separated into independent services for authentication, appointment management, AI integration, notifications, and scheduling. Stateless backend APIs support horizontal scaling through multiple application instances behind a load balancer. Frequently accessed data, such as available appointment slots, can be cached using Redis to reduce database load and improve response times.

Security

Security is implemented using industry-standard best practices. Authentication is based on JWT access tokens with role-based authorization for patients, doctors, and administrators. Passwords are securely hashed using bcrypt before storage. Sensitive credentials, including API keys and database passwords, are stored as environment variables instead of source code. Input validation prevents malformed requests, while HTTPS encrypts communication between clients and servers. Access to Google Calendar APIs is secured using OAuth 2.0, and application logs avoid storing confidential patient information to maintain privacy and regulatory compliance.

Overall, the proposed architecture delivers a reliable, secure, and scalable healthcare appointment platform capable of handling concurrent bookings, AI-assisted consultations, automated notifications, and future feature expansion while maintaining a seamless experience for patients, doctors, and administrators.